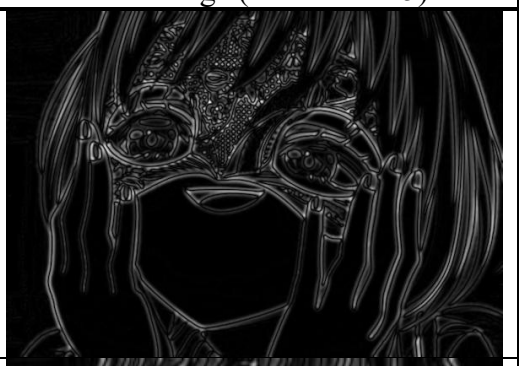
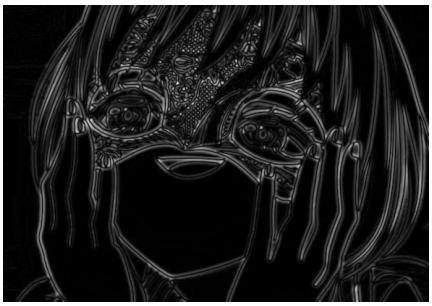
Computer Vision HW1 Report

Student ID: B11201024
Name: 蔡平樂

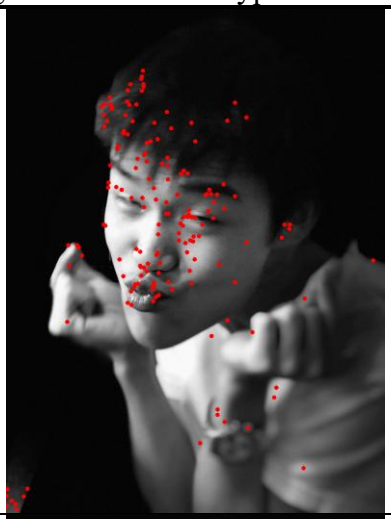
Part 1.

- **Visualize the DoG images of 1.png.**

	DoG Image (threshold = 5)		DoG Image (threshold = 5)
DoG1-1.png		DoG2-1.png	
DoG1-2.png		DoG2-2.png	
DoG1-3.png		DoG2-3.png	
DoG1-4.png		DoG2-4.png	

Images are normalized by $Img' := \frac{255}{\max(abs(Img))} \times abs(Img)$

- **Use three thresholds (1,2,3) on 2.png and describe the difference.**

Threshold	Image with detected keypoints on 2.png
0.5	
1.0	
5.0	

(describe the difference)

As the threshold increase, the number of feature points is decreasing, and more focus on the feature of the image.

Part 2.

- **Report the cost for each filtered image.**

Gray Scale Setting	Cost (1.png)
cv2.COLOR_BGR2GRAY	1207799
R*0.0+G*0.0+B*1.0	**1439568**
R*0.0+G*1.0+B*0.0	1305961
R*0.1+G*0.0+B*0.9	1393620
R*0.1+G*0.4+B*0.5	1279697
R*0.8+G*0.2+B*0.0	**1127913**

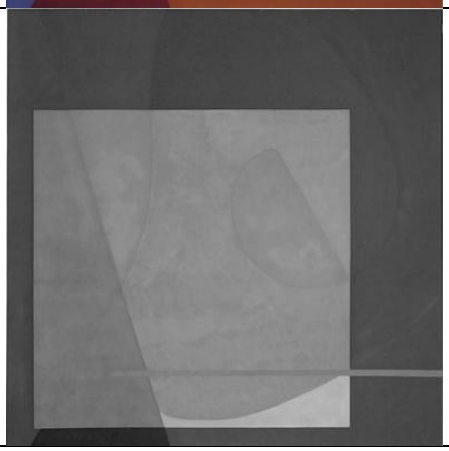
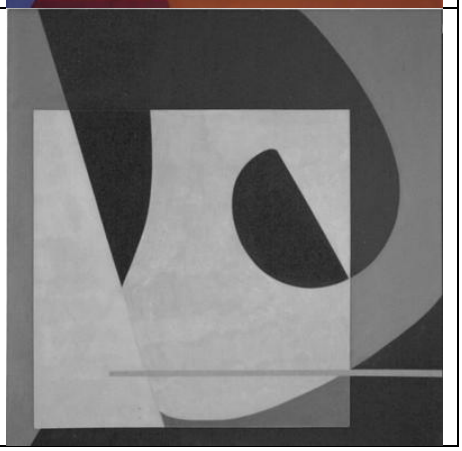
Gray Scale Setting	Cost (2.png)
cv2.COLOR_BGR2GRAY	183851
R*0.1+G*0.0+B*0.9	**77884**
R*0.2+G*0.0+B*0.8	86023
R*0.2+G*0.8+B*0.0	**188019**
R*0.4+G*0.0+B*0.6	128341
R*1.0+G*0.0+B*0.0	110862

- **Show original RGB image / two filtered RGB images and two grayscale images with highest and lowest cost.**

Original RGB image (1.png)	Filtered RGB image and Grayscale image of Highest cost	Filtered RGB image and Grayscale image of Lowest cost

(Describe the difference between those two grayscale images)

In the gray-scale image of the highest cost, both the ground and the leaf are dark, causing the filter hard to distinguish the leaf and the ground. In the gray-scale image of the lowest cost, the ground is dark but the leaf is light in contrast, encouraging the filter to attribute larger weight on the region of same color and smaller weight on the region of different color, which make the filtered image clear.

Original RGB image (2.png)	Filtered RGB image and Grayscale image of Highest cost	Filtered RGB image and Grayscale image of Lowest cost
		

(Describe the difference between those two grayscale images)

Similar as result in 1.jpg, the gray-scale image of the highest cost mixed the grange and purple, causing the filter cannot distinguish the region of different color. The gray-scale image of the smallest cost separate colors clearly, which make the filter perform better weighting.

- **Describe how to speed up the implementation of bilateral filter.**

由於 numpy 底層由 c++實作，對於相同的矩陣操作(如矩陣相加、常量相乘)，使用 numpy ndarray 操作能夠大幅提升速度，因此加速的方式為盡量使用矩陣操作進行計算。
比起原始使用 pixelwise 的計算，我們以偏移量進行分類操作。
可以發現，在偏移量固定為dy, dx時，每個 pixel I_{xy}的 spatial kernel 都相同，且 range kernel 都是與$I_{x-dx,y-dy}$的差。我們能夠以整個矩陣為單位計算這兩者，因此能封裝為單一矩陣操作，透過呼叫 numpy 大幅節省運算時間。 除此之外，我也透過 gemini 改寫程式碼，在迴圈外宣告陣列以減少記憶體重複宣告。

上述方式實作後，以 10 次執行平均後得出對於 ex.png 的處理速度約為 658.9ms

另外由於(dx, dy)與$(-dx, -dy)$實際上是以同樣的方式計算矩陣差，因此調整計算的 index 後便可在同一個 iteration 進行計算。使用上述方式能將迴圈數在減少兩倍，然而實際執行 10 次平均後得出速度為620.26ms，僅加快約38ms。